

Механизм внимания (attention, self-attention, @salyamq)

Механизм внимания (Attention) – это метод в искусственном интеллекте, который позволяет нейросети динамически определять, какие части входных данных наиболее важны для текущей задачи.

Он работает через вычисление весов важности для разных элементов входа: более важные элементы получают больший вес, а менее важные – меньший. Затем модель формирует взвешенную сумму представлений, создавая новый контекстный вектор.

Self-attention, в свою очередь, помогает модели понимать, как разные элементы входных данных связаны между собой. Например, как разные части информации взаимодействуют и влияют друг на друга в общем контексте. Этот механизм обеспечивает логическую связность и целостное понимание всей структуры данных

Как работает self-attention(Теория)

Пусть входная последовательность представлена матрицей: $X \in \mathbb{R}^{n \times d}$,

n – число токенов, d – размерность эмбединга.

Из X получаем три матрицы **Query (Q)**, **Key (K)** и **Value (V)**, где

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V \text{ где } W^Q, W^K, W^V \in \mathbb{R}^{d \times d_k}$$

Attention scores (оценка важности)

Считаем "похожесть" между токенами:

$$A_{ij} = \frac{Q_i K_j^T}{\sqrt{d_k}}$$

$A \in \mathbb{R}^{n \times n}$, элемент A_{ij} показывает, насколько i -й токен "смотрит" на j -й.

Нормализуем A , $N = \text{softmax}(A_{ij})$, каждая строка N – распределение вероятностей внимания.

Тогда получаем

$$\text{Attention}(Q, K, V) = AV$$

То есть,

$$\text{output}_i = \sum_{j=1}^n A_{ij} V_j$$

Как работает self-attention (Пример)

Допустим есть предложение "Карина идет в магазин".

Мы не можем работать с буквами напрямую, поэтому представим текст в виде токенов. Для упрощения будем считать, что 1 токен = 1 слово.

Эмбединги

Токенизация. Разбиваем фразу на части: ["карина", "идет", "в", "магазин"]. Здесь $n = 4$ (число токенов).

Эмбединг. Каждый токен заменяется вектором размерности d . Для упрощения, допустим что:

слово	вектор
Карина	[1, 0]
идет	[0, 1]
в	[1, 1]
магазин	[0.5, 1]

Position Encoding (позиционное кодирование). К векторам добавляется информация о позиции, чтобы модель знала, что «карина» стоит на 1-м месте, а «магазин» на 4-м.

Преобразования

Мы получили:

$$X_{4 \times 2} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ 0.5 & 1 \end{bmatrix}$$

Чтобы не утонуть в матрицах, возьмём: $W^Q = W^K = W^V = I$, тогда $Q = K = V = X$

(однако надо понимать, что матрицы W^Q, W^K, W^V это обучаемые параметры слоя).

Оценка важности

$$A = \frac{QK^T}{\sqrt{d_k}}$$

Мы имеем $d = 2$ (т.к. размерность векторов эмбединга равна 2). Тогда $\sqrt{d} \approx 1.41$

$$QK^T = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ 0.5 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 1 & 0.5 \\ 0 & 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 & 0.5 \\ 0 & 1 & 1 & 1 \\ 1 & 1 & 2 & 1.5 \\ 0.5 & 1 & 1.5 & 1.25 \end{bmatrix}$$

Делим на $\sqrt{d} \approx 1.41$

$$A = \frac{1}{1.41} \begin{bmatrix} 1 & 0 & 1 & 0.5 \\ 0 & 1 & 1 & 1 \\ 1 & 1 & 2 & 1.5 \\ 0.5 & 1 & 1.5 & 1.25 \end{bmatrix} \approx \begin{bmatrix} 0.71 & 0 & 0.71 & 0.35 \\ 0 & 0.71 & 0.71 & 0.71 \\ 0.71 & 0.71 & 1.42 & 1.06 \\ 0.35 & 0.71 & 1.06 & 0.89 \end{bmatrix}$$

Дальше применяем softmax, где

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$$

Важно понимать, что softmax применяется к каждой строке отдельно. Исходная 1 строка, где $x_1 = [0.71 \ 0 \ 0.71 \ 0.35]$, тогда подставив в $\text{Softmax}(x_i)$ получим $[0.31 \ 0.15 \ 0.31 \ 0.22]$. Что это означает для «Карина»? Теперь модель знает, что при формировании смысла слова «Карина» в этом предложении: 31% информации она берет из самой себя, 15% — из слова «идет» и так далее. Подставив каждую строку, получим матрицу

$$N = \text{Softmax}(A) \approx \begin{bmatrix} 0.31 & 0.15 & 0.31 & 0.22 \\ 0.14 & 0.29 & 0.29 & 0.29 \\ 0.18 & 0.18 & 0.37 & 0.26 \\ 0.16 & 0.23 & 0.33 & 0.28 \end{bmatrix}$$

Последний этап это умножение матрицы весов (результат softmax) на саму информацию, которую несут слова, то есть на матрицу Value (V).

$$\text{output} = N \times V$$

$$\text{Output} = \begin{bmatrix} 0.31 & 0.15 & 0.31 & 0.22 \\ 0.14 & 0.29 & 0.29 & 0.29 \\ 0.18 & 0.18 & 0.37 & 0.26 \\ 0.16 & 0.23 & 0.33 & 0.28 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ 0.5 & 1 \end{bmatrix} = \begin{bmatrix} 0.73 & 0.68 \\ 0.57 & 0.87 \\ 0.68 & 0.81 \\ 0.63 & 0.84 \end{bmatrix}$$

Матрица 'Output' это новые эмбединги слов после self-attention, то есть

токен	было	стало
Карина	[1, 0]	[0.73, 0.68]
идет	[0, 1]	[0.57, 0.87]
в	[1, 1]	[0.68, 0.81]
магазин	[0.5, 1]	[0.63, 0.84]



Каждый вектор после self-attention уже не описывает слово само по себе, а описывает его с учётом всех остальных слов в предложении. Например, слово «идет» в контексте «Карина идет в магазин» получает представление, связанное с физическим перемещением человека. А в предложении «Время идет быстро» это же слово будет иметь другое контекстное представление, связанное с течением времени и изменением состояния. То есть модель не хранит разные «значения» слова отдельно, а каждый раз формирует новое представление слова на основе контекста.

После self-attention каждый токен становится новой точкой в новом пространстве и эта точка уже зависит от контекста.

Self-attention на Pytorch

```
import torch
import torch.nn as nn

class SelfAttention(nn.Module): # класс self-attention слоя
    def __init__(self, embed_dim): # embed_dim = размер эмбединга
        super().__init__() # инициализация nn.Module
        self.embed_dim = embed_dim # сохраняем размерность

        self.q_linear = nn.Linear(embed_dim, embed_dim) # слой для Query
        self.k_linear = nn.Linear(embed_dim, embed_dim) # слой для Key
        self.v_linear = nn.Linear(embed_dim, embed_dim) # слой для Value
```

```

self.scale = embed_dim ** 0.5 # коэффициент масштабирования (sqrt(d))

def forward(self, x):
    """
    x: (batch_size, seq_len, embed_dim)
    """
    Q = self.q_linear(x) # (B, T, D) – Query
    K = self.k_linear(x) # (B, T, D) – Key
    V = self.v_linear(x) # (B, T, D) – Value

    # attention scores
    scores = torch.bmm(Q, K.transpose(1, 2)) # (B, T, T) – QK^T
    scores = scores / self.scale # нормализация (stability)

    attn = torch.softmax(scores, dim=-1) # вероятности внимания

    out = torch.bmm(attn, V) # взвешенная сумма V

    return out # выход

```

Cross-Attention, Multi-Head Attention

Cross-Attention

Cross-attention - это механизм внимания, где запросы (Q) берутся из одной последовательности, а ключи (K) и значения (V) - из другой. Cross-attention считается так же, как self-attention, но Q берется из другой последовательности.

$Q = X_{dec}W^Q$, $K = X_{enc}W^K$, $V = X_{enc}W^V$, дальше

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Что меняется по сравнению с self-attention:

компонент	self-attention	cross-attention
Q	X	X_dec
K	X	X_enc
V	X	X_enc

Cross-Attention в Pytorch

```

import torch
import torch.nn as nn

class CrossAttention(nn.Module): # класс cross-attention слоя
    def __init__(self, embed_dim): # embed_dim = размер эмбединга
        super().__init__() # инициализация nn.Module
        self.embed_dim = embed_dim # сохраняем размерность

        self.q_linear = nn.Linear(embed_dim, embed_dim) # слой для Query (из декодера)
        self.k_linear = nn.Linear(embed_dim, embed_dim) # слой для Key (из энкодера)
        self.v_linear = nn.Linear(embed_dim, embed_dim) # слой для Value (из энкодера)

        self.scale = embed_dim ** 0.5 # коэффициент масштабирования (sqrt(d))

    def forward(self, x_dec, x_enc):
        """
        x_dec: (batch_size, seq_len_q, embed_dim) – запрос
        x_enc: (batch_size, seq_len_kv, embed_dim) – контекст
        """
        Q = self.q_linear(x_dec) # (B, T_q, D) – Query из декодера
        K = self.k_linear(x_enc) # (B, T_kv, D) – Key из энкодера
        V = self.v_linear(x_enc) # (B, T_kv, D) – Value из энкодера

        # attention scores
        # (B, T_q, D) x (B, D, T_kv) -> (B, T_q, T_kv)
        scores = torch.bmm(Q, K.transpose(1, 2)) # QK^T – сопоставляем запрос с контекстом
        scores = scores / self.scale # нормализация (stability)

        attn = torch.softmax(scores, dim=-1) # вероятности внимания (на что смотрим в энкодере)

```

```

out = torch.bmm(attn, V) # взвешенная сумма V из энкодера

return out # выход (контекстный вектор для декодера)

```

Multi-Head Attention

Multi-Head Attention (многоголовое внимание) — это механизм, который запускает несколько attention «параллельно», чтобы модель могла смотреть на последовательность под разными углами. Если в одной голове attention модель нашла связь «подлежащее → сказуемое», то другая голова может поймать, например, «объект → действие» или дальнюю зависимость через несколько слов.

Идея простая: вместо одного набора Q, K, V мы делаем h разных наборов (голов), у каждой свои матрицы весов:

$$Q^{(i)} = XW_Q^{(i)}, \quad K^{(i)} = XW_K^{(i)}, \quad V^{(i)} = XW_V^{(i)}$$

где $i = 1..h$.

Дальше каждая голова считает обычный scaled dot-product attention:

$$\text{head}_i = \text{Attention}(Q^{(i)}, K^{(i)}, V^{(i)}) = \text{softmax}\left(\frac{Q^{(i)}K^{(i)T}}{\sqrt{d_k}}\right)V^{(i)}$$

После этого мы просто «склеиваем» результаты всех голов по последнему измерению и снова линейно преобразуем:

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

где W^O - ещё одна обучаемая матрица.

Multi-Head Attention на PyTorch (простая реализация)

```

import torch
import torch.nn as nn

class MultiHeadSelfAttention(nn.Module):
    def __init__(self, embed_dim, num_heads):
        super().__init__()
        assert embed_dim % num_heads == 0

        self.embed_dim = embed_dim
        self.num_heads = num_heads
        self.head_dim = embed_dim // num_heads

        self.q = nn.Linear(embed_dim, embed_dim)
        self.k = nn.Linear(embed_dim, embed_dim)
        self.v = nn.Linear(embed_dim, embed_dim)
        self.out = nn.Linear(embed_dim, embed_dim)

        self.scale = self.head_dim ** 0.5

    def forward(self, x):
        # x: (B, T, D)
        B, T, D = x.shape

        Q = self.q(x) # (B, T, D)
        K = self.k(x)
        V = self.v(x)

        # разбиваем на головы: (B, T, h, d_h) -> (B, h, T, d_h)
        Q = Q.view(B, T, self.num_heads, self.head_dim).transpose(1, 2)
        K = K.view(B, T, self.num_heads, self.head_dim).transpose(1, 2)
        V = V.view(B, T, self.num_heads, self.head_dim).transpose(1, 2)

        # scores: (B, h, T, T)
        scores = torch.matmul(Q, K.transpose(-2, -1)) / self.scale
        attn = torch.softmax(scores, dim=-1)

        # head outputs: (B, h, T, d_h)
        head_out = torch.matmul(attn, V)

        # склеиваем головы обратно: (B, T, D)
        head_out = head_out.transpose(1, 2).contiguous().view(B, T, D)

```

```
return self.out(head_out)
```

Примечания:

<https://arxiv.org/abs/1706.03762>

<https://habr.com/ru/articles/781770/>

https://en.wikipedia.org/wiki/Attention_Is_All_You_Need

[https://neerc.ifmo.ru/wiki/index.php?](https://neerc.ifmo.ru/wiki/index.php?title=%D0%9C%D0%B5%D1%85%D0%B0%D0%BD%D0%B8%D0%B7%D0%BC_%D0%B2%D0%BD%D0%B8%D0%BC%D0%B0%D0%BD%D0%B8%D1%8F)

[title=%D0%9C%D0%B5%D1%85%D0%B0%D0%BD%D0%B8%D0%B7%D0%BC_%D0%B2%D0%BD%D0%B8%D0%BC%D0%B0%D0%BD%D0%B8%D1%8F](https://neerc.ifmo.ru/wiki/index.php?title=%D0%9C%D0%B5%D1%85%D0%B0%D0%BD%D0%B8%D0%B7%D0%BC_%D0%B2%D0%BD%D0%B8%D0%BC%D0%B0%D0%BD%D0%B8%D1%8F)

@salyamq